

Power Query (M) – Data Cleaning & Transformation Script

Telecom Customer Churn Dashboard | Power BI Data Preparation Layer

PROJECT	Telecom Customer Churn Dashboard
SOURCE FILE	customer_churn_1M.csv
TARGET TABLE	customer_churn_1M
APPLIED IN	Power BI Desktop — Power Query Editor (Advanced Editor)
PREPARED BY	Laxman Ram
LAST UPDATED	August 2026

Overview

This query runs in the Power Query Editor and forms the data preparation layer for the churn dashboard. It loads the raw customer_churn_1M.csv export, removes duplicate and incomplete records, standardizes text and data types, and adds two derived columns – SeniorCitizenLabel and TenureGroup – that are used directly in the report visuals and slicers. The steps below follow the order in which they run inside the Advanced Editor.

1 Load Source Data

Imports the raw CSV using UTF-8 encoding and comma delimiters. This is the entry point of the query – every later step builds on this table.

```
Source =  
    Csv.Document(  
        File.Contents("customer_churn_1M.csv"),  
        [  
            Delimiter = ",",  
            Encoding = 65001,  
            QuoteStyle = QuoteStyle.Csv  
        ]  
    ),
```

2 Promote First Row as Headers

Converts the first row of the CSV into proper column headers, so fields such as customerID, gender, Contract, and Churn are recognized as column names rather than data.

```
#"Promoted Headers" =  
    Table.PromoteHeaders(  
        Source,  
        [PromoteAllScalars = true]  
    ),
```

3 Remove Duplicate Rows

Strips out exact duplicate customer records using `Table.Distinct`, guarding against the same customer appearing more than once in the source export.

```
#"Removed Duplicates" =  
    Table.Distinct(  
        #"Promoted Headers"    ),
```

4 Trim Whitespace from Text Columns

Removes stray leading and trailing spaces from key text fields. Untrimmed values are a common cause of broken filters and mismatched groupings later in the model.

```
#"Trimmed Text" =  
    Table.TransformColumns(  
        #"Removed Duplicates",  
        {  
            {"customerID", Text.Trim, type text},  
            {"gender", Text.Trim, type text},  
            {"Partner", Text.Trim, type text},  
            {"Dependents", Text.Trim, type text},  
            {"Contract", Text.Trim, type text},  
            {"PaymentMethod", Text.Trim, type text},  
            {"Churn", Text.Trim, type text}  
        }  
    ),
```

5 Replace Blank TotalCharges with 0

New customers with zero tenure sometimes have a blank `TotalCharges` value in the source file. Replacing blanks with "0" here avoids null errors once the column is converted to a numeric type in the next step.

```
#"Replaced Blank TotalCharges" =
    Table.ReplaceValue(
        #"Trimmed Text",
        "",
        "0",
        Replacer.ReplaceValue,
        {"TotalCharges"}
    ),
```

6 Correct Data Types

Explicitly sets the data type for every column – dates, whole numbers, decimals, and text – so Power BI handles aggregations, sorting, and relationships correctly.

```
#"Changed Type" =
    Table.TransformColumnTypes(
        #"Replaced Blank TotalCharges",
        {
            {"customerID", type text},
            {"gender", type text},
            {"SeniorCitizen", Int64.Type},
            {"Partner", type text},
            {"Dependents", type text},
            {"marital_status", type text},
            {"education", type text},
            {"age_group", type text},
            {"signup_date", type date},
            {"tenure", Int64.Type},
            {"Contract", type text},
            {"PaymentMethod", type text},
            {"late_payments", Int64.Type},
            {"MonthlyCharges", type number},
            {"TotalCharges", type number},
            {"Churn", type text}
        }
    ),
```

7 Add SeniorCitizenLabel

Derives a readable "Yes" / "No" label from the 0/1 SeniorCitizen flag, so the field can be dropped straight into visuals and slicers without a separate DAX measure.

```
#"Added SeniorCitizenLabel" =
    Table.AddColumn(
        #"Changed Type",
        "SeniorCitizenLabel",
        each
            if [SeniorCitizen] = 1
            then "Yes"
            else "No",
        type text
    ),
```

8 Add TenureGroup

Buckets the numeric tenure column into five readable bands, enabling cohort-style churn analysis by customer lifetime instead of raw month counts.

```
#"Added TenureGroup" =
    Table.AddColumn(
        #"Added SeniorCitizenLabel",
        "TenureGroup",
        each
            if [tenure] <= 12 then "0-1 yr"
            else if [tenure] <= 24 then "1-2 yr"
            else if [tenure] <= 48 then "2-4 yr"
            else if [tenure] <= 60 then "4-5 yr"
            else "5+ yr",
        type text
    ),
```

9 Remove Rows with Missing Customer IDs

Final safeguard step: discards any row without a valid customerID, so every remaining record can be uniquely identified in the data model.

```
#"Removed Missing Customer IDs" =
    Table.SelectRows(
        #"Added TenureGroup",
        each
            [customerID] <> null
            and Text.Trim([customerID]) <> ""
    )

in
    #"Removed Missing Customer IDs"
```

Summary of Applied Steps

#	Step	Purpose
1	Load Source Data	Load the raw CSV export into Power Query
2	Promote First Row as Headers	Turn the first row into column headers
3	Remove Duplicate Rows	Drop exact duplicate customer records
4	Trim Whitespace from Text Columns	Clean whitespace from text fields
5	Replace Blank TotalCharges with 0	Fill blank TotalCharges values with 0
6	Correct Data Types	Set correct data types on every column
7	Add SeniorCitizenLabel	Add a readable senior-citizen label
8	Add TenureGroup	Bucket tenure into cohort bands
9	Remove Rows with Missing Customer IDs	Drop rows with no customerID

Note: this transformation runs once, at refresh time, inside Power Query — no changes are made to the original CSV file. The cleaned output table (customer_churn_1M) feeds the data model and DAX measures used across the churn dashboard.